THAM LUANG CAVE RESCUE: A REINFORCEMENT LEARNING APPROACH

CS238/AA228, WINTER 2023

Kiran Bhat
Department of Computer Science
Stanford University
kvbhat@stanford.edu

Jeffery Shen
Department of Computer Science
Stanford University
jshen7@stanford.edu

March 17, 2023

ABSTRACT

We aim to train a reinforcement learning agent to perform a rescue of a group of people trapped in a flooding cave system. This idea was inspired by the Tham Luang cave rescue – a 2018 mission to save a junior football team trapped for over 2 weeks in a flooded cave in Thailand. [NatGeo, 2021]

To simulate the cave environment, we simplify a real-world 3D cave into a 2D gridworld representation. This problem represents a Markov Decision Process (MDP), with various sources of uncertainty. We explore this problem by utilizing and comparing multiple different RL learning algorithms including Q-learning, SARSA, discrete value iteration, and Monte Carlo Tree Search to solve the task of finding an optimal solution to the cave rescue problem.

1 Introduction

In 2018, a junior football team consisting of 12 children and their coach entered the Tham Luang Cave in Chiang Rai Province in northern Thailand. Shortly after they entered, heavy rainfall partially flooded the cave system, blocking their way out, and trapping them deep within. Efforts to locate the group were hampered by rising water levels and strong currents; and no contact was made for more than a week.

The point where the boys became stranded was about 2.5 mi from the entrance, and the route to the trapped team had several flooded sections, some with strong currents and zero visibility. These factors made the rescue seem almost impossible – at this point, the team had been stuck in the cave for 9 days, and in declining health.

In our project, we seek to train an agent using reinforcement learning techniques to "rescue" trapped rescuees in a simulated cave environment. We will simulate a cave environment using a 2D gridworld representation, where the cave contains things such as walls, water, and rescuees to be picked up. We include many sources of uncertainty, including uncertainty in what direction the agent moves after executing a move and differing amounts of health for the rescuees. Our simulated environment will represents a MDP – the MDP environment will assume that the agent has complete certainty about its current state. We will explore multiple RL algorithms including Q-learning, SARSA, discrete value iteration, and Monte Carlo Tree Search with the goal of analyzing which method most efficiently and successfully rescues the agent.

Why is this approach appropriate? While mazesolver/pathfinding algorithms already exist, these algorithms are unusable in the face of uncertainty. The field of reinforcement learning is very applicable to problems that are complex and that involve dynamic environments. RL methods learn by trial and error and receive feedback through rewards, which make them particularly useful for problems where the environment is complex. Additionally, RL agents have flexibility in their ability to handle changing environments as they can continuously update their policies based on feedback – something that static mazesolver algorithms cannot do.

2 Literature Review

We know that RL methods are able to solve maze environments, as described by Panov et al. [2018] in "Grid Path Planning with Deep Reinforcement Learning: Preliminary Results". In part, our cave environment problem is essentially a maze problem: the agent must find the optimal path to the "goal" (an exit in the maze problem, but a rescuee in our problem), and then find a path back to the start. Panov finds that while paths found by RL methods may sometimes be worse than traditional path-finding algorithms such as A*, the model in their paper (a neural network implementation of Q-network) learns to plan paths after the learning phase, and is capable of solving previously unseen instances.

One downside of RL models is that training can be extremely slow to converge, especially for model-free methods such as Q-learning and SARSA, with sparse rewards. A potential solution is utilizing eligibility traces, as described by Sutton [1988]. Q-learning and SARSA can both be modified to propagate rewards backwards to the state/action pairs that lead to the source of the reward using eligibility traces, where the credit is decayed exponentially for states further away from the reward.

These methods rely on exploration to successfully backpropagate rewards/Q-values to the remainder of the state space, which given given infinite timesteps, these methods are generally guaranteed to converge. But, this becomes exponentially harder as the state and action space become larger. Another potential solution is described in Tizhoosh [2006], where Tizhoosh proposes using Opposition-Based learning, a algorithm built on Q learning that conducts updates by considering states/actions and opposite states/actions simultaneously, which can shorten the exploration period needed for convergence.

But, because Q-learning assumes that the state is known with absolute certainty at each timestep, base Q-learning cannot be effectively used to solve POMDPs. Instead, Deep Q-learning exists for POMDPS, as described by Hausknecht and Stone [2017] in "Deep Recurrent Q-Learning for Partially Observable MDPs." In the case of Deep Q-Learning, the model is a neural network parameterized by weights and biases collectively denoted as θ. Q-values are estimated online by querying the output nodes of the network after performing a forward pass given a state input, denoted by $Q(s, a|\theta)$. Instead of updating individual Q-values as is the case in base Q-learning, updates are now made to the parameters of the network to minimize a differentiable loss function. θ allows for the model to generalize beyond only the state action pairs that it has seen before, which makes this able to solve a POMPDP such as an Atari game where the model is given frames of a game being played where the velocity/direction of moving objects are not revealed, and thus the state is only partially observable.

3 Background

In this paper, we considered several methods of solving our cave system problem. Specifically, we used the Online and Offline methods as described in our introduction, with descriptions below. We also utilized the Julia POMDPS package for our reinforcement learning algorithms. [Egorov et al., 2017].

3.1 Offline Methods

Offline methods involve making decisions based on a pre-determined plan or strategy that is developed prior to experiencing the uncertainty.

3.1.1 Q Learning

Q-learning involves applying incremental estimation of the action value function $Q(s, a)$. The incremental update rule to estimate the action value function is represented by:

$$Q(s, a) \leftarrow Q(s, a) + \alpha(r + \gamma max_{a'} Q(s', a') - Q(s, a)) \tag{1}$$

Where α is the learning rate, γ is the discount rate, and s' and a' are the new states and action taken.

Q-learning is an off-policy method because it attempts to find the value of the optimal policy while following the exploration strategy.

3.1.2 SARSA

SARSA is an alternative to Q-learning, that shares a very similar update equation. This algorithm uses the actual next action a' to update Q instead of maximizing over all possible actions, with the equation:

$$Q(s, a) \leftarrow Q(s, a) + \alpha(r + \gamma Q(s', a') - Q(s, a)) \tag{2}$$

After many iterations and with a valid exploration policy, a' will converge to $argmax_{a'}Q(s', a')$. SARSA is referred to as a type of on-policy reinforcement learning method because it attempts to directly estimate the value of the exploration policy as it follows it.

We will also utilize eligibility traces to implement SARSA(λ). Eligibility traces involve propagating reward backward to the states and actions leading to the source of the reward. To do so, we maintain an exponentially decaying visit count $N(s, a)$ for all state-action pairs. The SARSA update is then partially applied to every state-action pair according to this decaying visit count. Let δ represent the incremental update for SARSA (equation 2). Every entry in the action value function is then updated with the equation:

$$Q(s, a) \leftarrow Q(s, a) + \alpha\delta N(s, a) \tag{3}$$

3.1.3 Discrete Value iteration

Value iteration works by updating the value function itself directly, begining with any bounded value function $U(s)$. In the Julia POMPDPs library that we use, the value function is initialized to all zeros, i.e $U(s) = 0$ for all s. Value iteration is a good solution because repeated application of the Bellman update is guaranteed to converge to the optimal value function.

In each iteration, the value function can be improved by applying the Bellman Update:

$$U_{k+1}(s) = max_a(R(sa, a) + \gamma \sum_{s'} T(s'|s, a)U_k(s')) \tag{4}$$

We can extract the best policy (greedily) from the value function using the following equation that maximizes the lookahead equation:

$$\pi(s) = argmax_a(R(s, a) + \gamma \sum_{s'} T(s'|s, a)U(s')) \tag{5}$$

3.2 Online Methods

Offline methods can potentially be intractable in high-dimensional problems. To combat this, online methods find actions based on states that are reachable from the current state, which is usually orders of magnitude smaller than the full state space.

3.2.1 Monte Carlo Tree Search

Monte Carlo Tree Search (MCTS) is an example of online planning – it works by building a search tree where each node represents a given state, and every edge represents a possible action. At every iteration, MCTS runs *m* simulations from the current state and updates estimates of the action value function, as well as a record of the number of times a state/action pair has been selected. After all *m* simulations, the action that maximizes Q(s, a) is chosen.

MCTS uses both exploration and exploitation to guide the search towards optimal parts of the search tree. Our implementation selects the next node to expand upon based on selecting the action that maximizes the UCB1 exploration heuristic, which is:

$$Q(s, a) + c\sqrt{\frac{logN(s)}{N(s, a)}} \tag{6}$$

where N(s) is the total number of times state s has been visited, and c is an exploration parameter.

4 Environment

This section will describe how we will define a Markov decision process that represents a cave environment with a diver agent and a target that need to be rescued.

4.1 State Space

We represent a state of the cave environment as the x and y positions of the diver, as well as the boolean value of whether or not the diver has picked up the rescuee. Our state space consists of all the possible combinations of x positions and y positions (which depends on the size of the cave), and boolean values. We found that both caves were small enough that we could represent both state spaces in their entirety, rather than using a generative model.

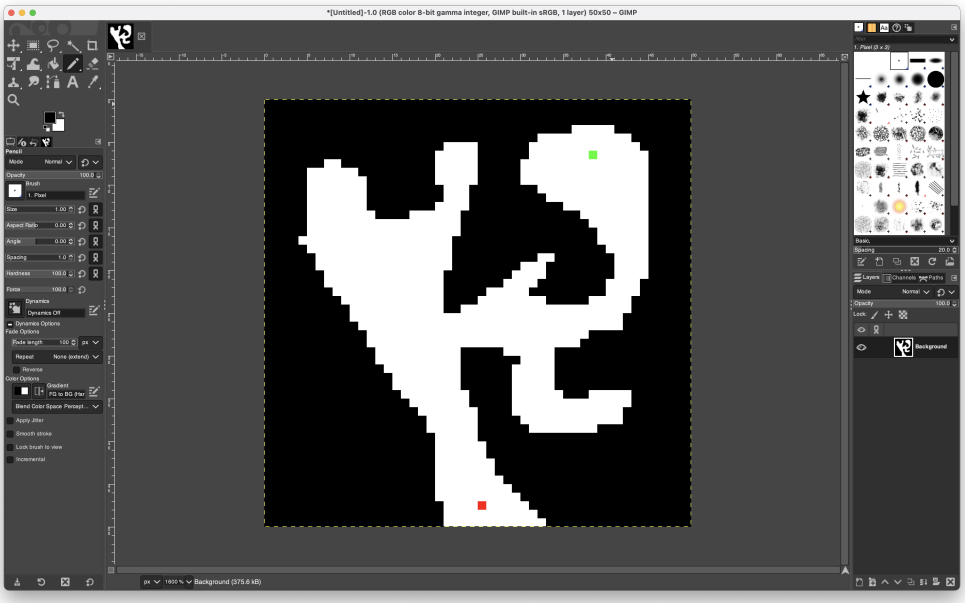


Figure 1: Example cave system drawn in GIMP. Black pixels indicate cave walls, white pixels indicate empty space, red pixels indicate the diver, and green pixels indicate the rescuee.

4.2 Action Space

Since we modeled our cave with a 2d grid, the action space for our diver agent consists of moving North, East, South, or West. The actions of picking up a diver and dropping them off at the rescue location are implicit to the transition/reward functions.

4.3 Transitions

In a real underwater cave system, a diver can often become disoriented and move in the wrong direction as a result. We decided to model this uncertainty within our transition function, by assigning a certain probability (p = 0.2) to moving in a different direction than the diver agent intends. However, if a diver is in the same position as the rescuee and hasn't yet picked up a rescuee, the transition function will deterministically provide the state in which the diver picked up that rescuee. The transition function will also deterministically provide the termination state when the diver reaches the rescue location with a rescuee.

4.4 Rewards

We designed our reward function in order to encourage our diver to efficiently pick up and rescue the rescuee. Our reward scheme is as follow:

- (-5) Punish the diver for not having a rescuee
- (-100) Punish the diver for running into a wall or going out of bounds
- (+1000) Reward the diver for picking up a rescuee
- (+10000) Reward the diver for bringing the rescuee to the rescue location

4.5 Generating the environment

We decided to use an open-source graphics editor called GIMP to draw cave systems, which we can then automatically load into a 2d numpy array to represent the cave walls. We can also obtain the initial position of the diver and the rescuees from the image. See Figure 1 for an example cave system drawn with GIMP.

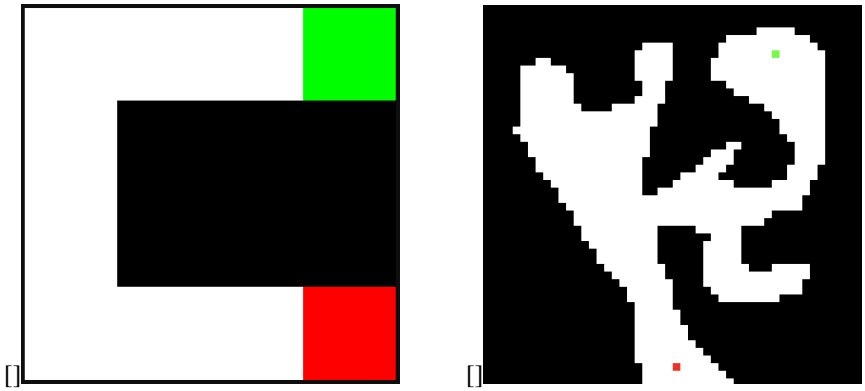


Figure 2: Small and large cave environments.

5 Solving the Cave Problem

To solve the cave problem, we used the POMDPs.jl Julia library Egorov et al. [2017] to define our cave as an MDP. To do so, we defined the state space, the action space, the transition function, the reward function, the discount rate (we chose 0.95), the initial state distribution, and a terminal state function, and use the QuickMDP functionality provided by POMDPs.jl.

After defining our MDP, we used a variety of offline and online solvers to compute a policy. We used the following solvers and parameters:

- ValueIterationSolver: Max iterations = 1000
- QLearningSolver: Number of episodes = 5000, Learning rate = 0.8, Exploration policy = Epsilon Greedy with Epsilon = 0.5
- SARSASolver: Same parameters as QLearningSolver
- MCTSSolver: Number of iterations = 50, Depth = 20, Exploration constant = 5.0

6 Results

We evaluated our policies on 2 different environments: a small cave and a large cave.

Small Cave Environment:

Method	Mean Episodic Reward
Discrete Value Iteration	10,955
Q-Learning	10,955
SARSA	10,855
SARSA(λ)	10,955
MCTS	9325
Random Policy	-1890.45

Table 1: This table describes the mean episodic reward for the small cave environment. The maximum score possible is 10,955. Our methods trained for 10 episodes with a maximum episode length of 30 steps.

Large Cave Environment:

Method	Mean Episodic Reward
Discrete Value Iteration	10,670
Q-Learning	-1,000
SARSA	-1,000
SARSA(λ)	-1,000
MCTS	-1,000
Random Policy	-1,000

Table 2: This table describes the mean episodic reward for the large cave environment. The maximum score possible is 10,670. Our methods trained for 10 episodes with a maximum episode length of 200 steps.

7 Analysis

RL methods have both strengths and weaknesses in gridworld/maze-like environments. Firstly, these RL methods usually result in slower and more inefficient paths than a pathfinding algorithm alone (such as A*). However, traditional pathfinding/mazesolving algorithms depend on the fact that the maze environment is completely static and obstacles positions are known beforehand. Reinforcement learning models have the benefit of being able to be either deterministic, or non-deterministic in their actions, which can lead to better results when the environment changes. In Q-learning, for example, each state contains a probability distribution for the possible actions to be taken. In a deterministic agent, the action with the highest probability is always selected; in a non-deterministic agent, an action is selected in accordance to the given probability distribution at the current state.

Our project also demonstrates one of the disadvantages of Q-learning and SARSA: that learning can be prohibitively slow. This is especially pronounced with sparse rewards – i.e, where the environment only has one or a few goal states with a large reward, and no reward for the other state/action pairs. These model-free methods also are weak in instances where the environment *itself* changes. If a gridworld is modified by just changing the location of the rescuees, a Q-learning agent trained on a prior gridworld will completely fail.

We found that discrete value iteration was the only one to result in convergence for both the easy cave example and the large cave example. As outlined above, one of the main problems with Q-learning and other model-free methods that rely on estimating the value function solely based on exploration is that the time required to converge to an optimal solution scales with the size of the state space. But, as outlined in our theory section, value iteration is guaranteed to converge upon the optimal value function.

But, why exactly is value iteration guaranteed to converge upon the optimal value function (and thus, also the optimal policy)? Value iteration guarantees optimality because it works by repeatedly applying the Bellman update equation to bring the value function closer to satisfying the Bellman optimality function. We believe that this is the reason why value iteration was the only method that successfully solved the large cave map – other algorithms overly rely on exploration to generate policies, which can result in learning instabilities and require huge amounts of exploration when the state/action space become large.

8 Conclusion and Future Work

Overall, we found that while all of our tested methods were able to solve the small cave environment, only discrete value iteration was able to successfully solve the large cave environment.

One area of this problem that could be explored more is the use of other solving methods on cave environments with very large state spaces. One such method our team considered was the use of local approximation value iteration, combined with the use of multilinear interpolation. We think the use of multilinear interpolation could be effective for approximating the state space of this particular problem, since proximity in state space corresponds with proximity in the location of the diver. Although we were not able to complete our implementation of this approach, it would be a very interesting area to look into further.

Additionally, another area to explore is modeling the cave environment as a Partially Observable MDP (POMDP) instead of a MDP. So, instead of the agent having completely certainty of its state in the environment, the agent would only have a belief of its state. This simulation would be closer to a real cave-rescue situation, where murky waters can make the diver unsure about their current location.

If we explored making this problem a POMDP instead, we would have to use alternative RL algorithms. As described above in the analysis, our methods that attempt to estimate the value function itself will fail when the agent's current state is uncertain. However, there are alternatives and other pathways to explore in this scenario. One example is Deep Q-learning, as mentioned in our literature review. [Hausknecht and Stone, 2017]

Finally, we could also expand upon the sources of uncertainty in our environment. One major source that we wanted to implement is rising water levels. We found that implementing it would create too much complexity in our current approach (and result in all of our models failing), so we decided to not include it in this project.

9 Contributions

Both partners worked together on the code and creation of our custom environment. In the writeup, Jeffery focused on the Intro/Abstract, Literature review, Background, and Analysis. Kiran focused on the Environment, Solving the Cave Problem, Results, and Conclusion.

References

NatGeo. National Geographic, 2021.

Aleksandr I. Panov, Konstantin S. Yakovlev, and Roman Suvorov. Grid path planning with deep reinforcement learning: Preliminary results. *Procedia Computer Science*, 123:347–353, 2018. ISSN 1877-0509. doi:https://doi.org/10.1016/j.procs.2018.01.054. URL https://www.sciencedirect.com/science/article/pii/S1877050918300553. 8th Annual International Conference on Biologically Inspired Cognitive Architectures, BICA 2017 (Eighth Annual Meeting of the BICA Society), held August 1-6, 2017 in Moscow, Russia.

Richard S. Sutton. Learning to predict by the methods of temporal differences. 3(1), 1988. ISSN 0885-6125. doi:10.1023/A:1022633531479. URL https://doi.org/10.1023/A:1022633531479.

Hamid Tizhoosh. Opposition-based reinforcement learning. *JACIII*, 10:578–585, 07 2006. doi:10.20965/jaciii.2006.p0578.

Matthew Hausknecht and Peter Stone. Deep recurrent q-learning for partially observable mdps, 2017.

Maxim Egorov, Zachary N. Sunberg, Edward Balaban, Tim A. Wheeler, Jayesh K. Gupta, and Mykel J. Kochenderfer. POMDPs.jl: A framework for sequential decision making under uncertainty. *Journal of Machine Learning Research*, 18(26):1–5, 2017. URL http://jmlr.org/papers/v18/16-300.html.